

Contents

AI Coding Cost Optimisation	1
Executive summary	1
The problem	1
When not to use AI	2
Token efficiency	2
Open Brain: the memory layer	5
Machine-specific setup	6
The workflow	6
What else to consider	6

AI Coding Cost Optimisation

A practical guide to cutting AI coding costs by 90 per cent without sacrificing quality.

July 2026

Executive summary

1. **Use free models for 80 per cent of work.** Reserve paid models for architecture, security, and production.
2. **Enable prompt caching.** Keep static content at the start of every prompt. Do not rearrange it between sessions.
3. **Send only what matters.** Use `@file` references. Be specific about line numbers. Omit unchanged code.
4. **Batch related work.** One request for three bugs, not three requests for one bug each.
5. **Set up Open Brain.** Forty-five minutes of setup saves 15-40 minutes per day of context reloading.
6. **Adopt model cascading.** Free first, paid only on failure.
7. **Use structured output.** JSON mode cuts parsing tokens by 30-50 per cent.
8. **Specialise agents.** Different models for planning, coding, and reviewing.

Total monthly cost: EUR 15-20. Previous spending: USD 100-300. The tools are free. The discipline is not.

The problem

AI coding assistants burn money fast. At heavy usage rates—roughly 13.5 million tokens per hour—a developer can blow through USD 50 worth of credits in under a day. Most of this spending is wasteful: re-loading context that the system already knows, using expensive models for trivial tasks, and failing to exploit free alternatives that have closed the quality gap to within 5-15 per cent of premium offerings.

The solution is not to spend less time with AI. It is to spend smarter.

When not to use AI

AI is not always the right tool. Using it for every task wastes tokens and time.

Skip AI for: one-line config changes; tasks you already know how to do; debugging when you have not read the error message; exploratory work where understanding the system yourself matters more than delegating it.

Use AI for: multi-file changes where pattern consistency matters; boilerplate generation; code review; tasks outside your primary language or framework.

The cheapest token is the one you never send.

Token efficiency

1. Prompt caching

Most providers cache the beginning of a prompt automatically. If your system instructions and file context appear first, they are served from cache on subsequent requests at 90 per cent lower cost.

How: Put static content at the start of every prompt. Do not rearrange context between sessions. The cache key is positional—moving content breaks it.

2. Context window management

Sending an entire repository when you need one file is like hiring a moving company to carry a book.

Bad: “Here’s my codebase: [paste 50 files]” — 10,000 tokens.

Good: “Fix the bug in `@src/auth/login.ts` line 42. The session cookie is not being validated.” — 2,000 tokens.

How: Before each request, ask: Am I sending only what the model needs? Use `@file` references. Omit unchanged files. Be specific about line numbers and symptoms.

3. Response length control

Models generate until they run out of tokens or hit a limit. Setting `max_tokens` based on task complexity prevents waste.

Task	max_tokens
Simple fix	500
Code review	1000
Feature plan	2000
Complex refactor	4000

In OpenCode, add to `~/.config/opencode/opencode.json`:

```
{  
  "agent": {  
    "build": {
```

```

    "max_tokens": 2000
  }
}
}

```

4. Structured output

Request JSON or YAML responses instead of free-form text. Structured output uses 30-50 per cent fewer tokens because the model does not generate filler sentences or explanatory prose. It also eliminates the token cost of parsing free-form responses.

How: Append “Respond as JSON only, no explanation” to your prompt. Use schema validation to catch errors automatically.

5. Model cascading

Try the cheapest model first. Escalate only when quality is insufficient.

When to escalate: - Complex architecture: DeepSeek-V4-Flash (USD 0.09 per MTok) - Security features: Qwen3-Coder-Flash (USD 0.20 per MTok) - Production deployment: MiMo-V2.5-Pro (USD 0.43 per MTok)

6. Failure detection

AI models hallucinate confidently. You need verification systems, not trust.

Hard signals (unambiguous failure): - Build fails. - Tests fail. - Linter catches errors. - Type checker rejects the code.

These are binary. The model failed. Escalate.

Soft signals (suspicious output): - Cross-model review catches issues. - Self-consistency check: ask the same question twice. If the answers diverge, the model is uncertain. - Hallucinated APIs: the model references functions or parameters that do not exist. - Overly confident tone. Models that say “this is definitely correct” are often wrong.

The verification gate:

1. Free model produces output.
2. Automated verification runs: build, lint, test.
3. If all pass, accept. If any fail, escalate.
4. On escalation, a different free model reviews and attempts to fix.
5. If that also fails, escalate to paid model.

A free model that passes all tests is more trustworthy than a paid model that fails them.

7. Choosing the right verification

Match the verification depth to the risk.

Task type	Risk	Verification approach
Typo fix	Trivial	Visual scan

Task type	Risk	Verification approach
Simple bug fix	Low	Lint + existing tests
New feature (single file)	Medium	Build + lint + targeted tests
New feature (multi-file)	High	Full build + lint + all tests + cross-review
Security change	Critical	Full build + lint + all tests + manual review
Database migration	Critical	Full build + lint + all tests + rollback plan

The rule of thumb: if the change could break production, verify it like it will. If it cannot break anything, skip verification and move on.

8. Worked example

A free model is asked to add a `POST /api/users` endpoint to an Express.js application. The task is high-risk: multi-file change, new database interaction, new API contract.

Step 1: Plan. Free model generates the plan. A different free model reviews it and catches a missing rate-limiting middleware.

Step 2: Implement. Free model writes three files following existing patterns.

Step 3: Verify. `make build` passes. `make lint` passes. Three of 12 tests fail: wrong status codes, missing error handling, and a password hash leak. Hard signal — do not commit.

Step 4: Fix. Model fixes the three failures. Re-run: all 12 tests pass.

Step 5: Review. The `@pr-review` agent finds that the password hash is returned in the error path for duplicate emails — something the tests missed. Fix and re-verify.

Step 6: Commit. All checks pass. Commit.

Stage	What was caught	Caught by
Plan review	Missing rate limiting	Cross-model review
Implementation	Wrong status codes, missing error handling, password leak	Automated tests
Code review	Password hash in error response	Cross-model review (security)

Without verification, all three issues would have reached production. Time cost: roughly three minutes. A paid model would have taken the same time but cost USD 0.50-2.00 instead of nothing.

9. Batching

Three separate bug-fix requests burn three sets of context-loading tokens. One request covering all three bugs saves 40-60 per cent.

Bad: Three requests: “Fix login.ts”, “Fix register.ts”, “Fix reset-password.ts”.

Good: One request: “Fix these three bugs: login.ts line 42 (session cookie), register.ts line 15 (missing validation), reset-password.ts line 28 (token expiry).”

10. Local preprocessing

On a desktop with a 2080 Ti, Ollama runs Qwen 2.5 Coder 7B at 30-50 tokens per second. Use it for code formatting, simple refactors, and test generation. Do not use it for complex architecture, multi-file changes, or security decisions.

11. Agent specialisation

Different models have different strengths. Using the same model for everything wastes its strengths and exposes its weaknesses.

Task	Model	Why
Planning	DeepSeek-V4-Flash Free	Fast, good at structure
Coding	DeepSeek-V4-Flash Free	Reliable, free
Reviews	Nemotron 3 Ultra Free	Different architecture, catches different bugs
Security	Qwen3-Coder-Flash (paid)	Stronger reasoning

Cross-model review is not optional. It catches semantic errors that same-model review misses.

12. Streaming for faster feedback

Enable streaming responses. You see output as it generates, not after a 30-second wait. This cuts perceived latency by 80 per cent and lets you spot bad output early — before the model finishes generating 4,000 tokens of wrong code.

How: Most tools enable streaming by default. If yours does not, check the settings.

13. Local linting before AI

Run your linter before sending code to AI. A model that receives clean code produces cleaner output. A model that receives code with lint errors often reproduces or compounds them.

How: `make lint` before every AI request. Ten seconds of local linting saves minutes of AI-generated fixes.

Open Brain: the memory layer

Context reloading is the single largest source of waste. Every new session re-explains project structure, conventions, and recent decisions. Open Brain — an open-source system that gives every AI tool the same persistent memory via vector search and MCP — solves this. You store decisions, patterns, and context once; every future session starts with the AI already knowing your project.

Setup time: 15-45 minutes depending on cloud or offline deployment. **Detailed guide:** see `open-brain-guide.md`.

Machine-specific setup

Chromebook

No GPU, no local models. Use OpenRouter exclusively. Set free models as default in the global config. Set up Open Brain for persistent memory.

Budget: USD 0-5 per day.

Desktop (i9-9900K / 2080 Ti)

Full GPU available. Use local Ollama for most work, OpenRouter as fallback. Set up Open Brain for persistent memory.

Budget: EUR 13 per month electricity plus USD 0-10 API.

Dual machine

Use the Chromebook for mobile work with free API models. Use the desktop for offline and privacy-sensitive tasks with local models. Open Brain syncs knowledge between both.

Combined cost: EUR 15-20 per month.

The workflow

1. **Start of session:** AI searches Open Brain for relevant context. No re-explaining.
2. **Planning:** Free model writes the plan. Different free model reviews it.
3. **Implementation:** Free model writes code. Send only relevant files via `@file` references.
4. **Verification:** Free model runs build/lint/tests. Escalate to paid model only on failure.
5. **Capture:** Store decisions, patterns, and debugging notes in Open Brain for next time.

Per-feature cost drops from USD 5-20 to effectively zero.

What else to consider

Pre-commit hooks. Run lint and type-check automatically before every commit. Catches issues before they reach the AI, saving tokens on both sides.

Git stash for experimentation. When trying a risky approach, `git stash` your work first. If the AI produces garbage, restore in seconds. This encourages experimentation without fear of losing work.

Diff-based edits. Prefer tools that send only the changed lines, not entire files. OpenCode's edit tool sends the diff, not the file. This cuts context tokens by 60-80 per cent for large files.

Conversation management. Start a new conversation for each distinct task. Long conversations degrade model performance and waste tokens re-loading old context. If you have been chatting for more than 20 minutes, start fresh.

Model rotation. Rotate between free models every few requests. This avoids rate limits and exposes you to different model strengths. DeepSeek-V4-Flash for coding, Nemotron for reviews, Gemma for variety.

Prompt templates. Write reusable prompts for common tasks: code review, bug fix, test generation, plan writing. Store them in your project. A good template saves 20-30 tokens per request and produces more consistent output.

Monitor your spending. Check your OpenRouter dashboard weekly. If you are spending more than USD 20 per month on paid models, you are escalating too often. Free models should handle 80 per cent of work.

Sources: OpenRouter pricing (June 2026), MiMo Code credit documentation, independent model benchmarks (LiveCodeBench, Coding Index, Agentic Index). Benchmark figures are approximate and sourced from public leaderboards at time of writing. Costs are in the currency quoted by each provider; EUR 1 is approximately USD 1.10 at time of writing.